

Introdução a Banco de Dados

Aula 03

Restringindo e Classificando Dados

Módulo 02 — Fundamentos de SQL

Professor: Júlio César • SENAC Entra21

O que veremos hoje

1 A cláusula WHERE

2 Condições de comparação

3 BETWEEN, IN e LIKE

4 Tratamento de NULL

5 Operadores AND, OR, NOT

6 Regras de precedência

7 Ordenação com ORDER BY

Cláusula WHERE — restringindo as linhas retornadas

A cláusula *WHERE* filtra quais linhas serão exibidas. Aparece logo após *FROM* e contém uma condição que cada linha deve satisfazer.

- Sintaxe: `SELECT` colunas `FROM` tabela `WHERE` condição;
- A condição é avaliada linha a linha
- Strings e datas vão entre aspas simples ' '
- Strings são sensíveis a maiúsculas e minúsculas
- Formato de data padrão Oracle: DD-MON-RR

```
SELECT last_name, job_id, department_id
FROM   employees
WHERE  department_id = 90;

SELECT last_name, hire_date
FROM   employees
WHERE  hire_date = '17-FEB-96';
```

Condições de comparação — testando valores

Comparações testam se um valor cumpre um critério em relação a outro. Funcionam com números, textos e datas. Alguns operadores têm escrita diferente da matemática.

Operador	Significado
=	Igual a
>	Maior que
>=	Maior ou igual a
<	Menor que
<=	Menor ou igual a
<> ou !=	Diferente de

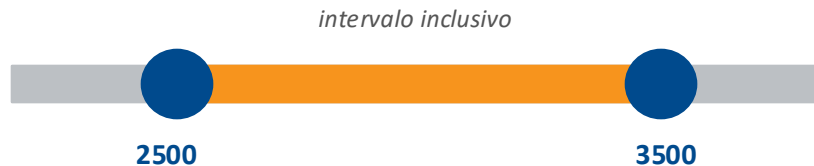
```
-- Funcionarios com salario ate 3000:
SELECT last_name, salary
FROM   employees
WHERE  salary <= 3000;

-- Sobrenome especifico (case sensitive):
SELECT last_name
FROM   employees
WHERE  last_name = 'King';
```

BETWEEN — testando valores em um intervalo

Verifica se um valor está dentro de uma faixa, incluindo os limites inferior e superior. Alternativa mais limpa do que combinar $\geq$ e $\leq$ com AND.

- Sintaxe: coluna BETWEEN min AND max
- Os limites fazem parte do intervalo
- Sempre informe o MENOR valor primeiro
- Funciona com números, datas e strings



```
SELECT last_name, salary
FROM employees
WHERE salary BETWEEN 2500 AND 3500;

-- Equivale a:
WHERE salary >= 2500 AND salary <= 3500;
```

IN — testando lista de valores

Verifica se um valor está presente em uma lista de opções. Substitui várias condições OR encadeadas, deixando o código muito mais legível.

Sem IN — verboso

```
SELECT last_name, manager_id
FROM   employees
WHERE  manager_id = 100
       OR manager_id = 101
       OR manager_id = 201;
```



Com IN — limpo

```
SELECT last_name, manager_id
FROM   employees
WHERE  manager_id IN (100, 101, 201);
```

- Sintaxe: coluna IN (val1, val2, val3, ...)
- Strings dentro do IN também ficam entre aspas simples
- Use NOT IN para excluir valores da lista

LIKE — buscando padrões em texto

LIKE filtra strings por padrões, e não por igualdade exata. Funciona com dois caracteres curinga (wildcards) — o sinal de porcentagem (%) e o sublinhado (_).

Curinga	Significado
%	Zero ou mais caracteres quaisquer
_	Exatamente um caractere qualquer

```
-- Sobrenomes que comecam com 'S':  
WHERE last_name LIKE 'S%';  
  
-- Segunda letra e 'o':  
WHERE last_name LIKE '_o%';  
  
-- Procurar o proprio caractere _:  
WHERE job_id LIKE 'SA\_%' ESCAPE '\';
```

IS NULL e IS NOT NULL — testando ausência de valor

NULL representa ausência de valor — não pode ser comparado com = ou <>. Para testar nulos, use IS NULL (verifica se está vazio) e IS NOT NULL (verifica se tem valor).

X Errado: `WHERE commission_pct = NULL`
Esta comparação NUNCA retorna verdadeiro!

✓ Correto: `WHERE commission_pct IS NULL`

```
-- Funcionarios SEM comissao:
SELECT last_name, job_id
FROM   employees
WHERE  commission_pct IS NULL;

-- Funcionarios COM comissao:
SELECT last_name, commission_pct
FROM   employees
WHERE  commission_pct IS NOT NULL;
```


Operadores lógicos — combinando condições

Combine várias condições com AND (todas verdadeiras), OR (basta uma) e NOT (inverte o resultado).

AND

Todas as condições devem ser verdadeiras

```
WHERE salary >= 5000  
      AND job_id = 'IT_PROG';
```

OR

Basta uma condição ser verdadeira

```
WHERE salary >= 10000  
      OR job_id = 'IT_PROG';
```

NOT

Inverte o resultado da condição

```
WHERE job_id NOT IN  
      ('ST_CLERK',  
       'IT_PROG');
```

Regras de precedência — quem vem primeiro?

Quando vários operadores aparecem juntos, o SGBD aplica uma ordem fixa. Conhecer essa ordem evita resultados inesperados.

- Operadores são avaliados em ordem hierárquica
- AND é avaliado ANTES de OR
- Parênteses () têm prioridade absoluta
- Quando houver dúvida, use parênteses — ganham legibilidade

Ordem	Operador
1	Operadores aritméticos (*, /, +, -)
2	Concatenação ()
3	Comparações (=, <, >, <=, >=, <>)
4	IS [NOT] NULL, LIKE, [NOT] IN
5	[NOT] BETWEEN
6	NOT
7	AND
8	OR

ORDER BY — classificando o resultado

ORDER BY define a ordem em que as linhas aparecem. É sempre a última cláusula da consulta. Por padrão, a ordenação é crescente (ASC); use DESC para inverter.

- Sintaxe: ORDER BY coluna [ASC | DESC]
- ASC é o padrão (geralmente implícito)
- DESC inverte: maior → menor, Z → A
- Pode ordenar por nome, apelido (alias) ou posição
- Várias colunas: separe por vírgula — a 1ª tem prioridade

```
-- Crescente (padrao):  
SELECT last_name, salary FROM employees  
ORDER BY salary;  
  
-- Decrescente:  
ORDER BY salary DESC;  
  
-- Por apelido de coluna:  
SELECT employee_id, salary*12 AS anual  
FROM employees  
ORDER BY anual;  
  
-- Varias colunas:  
ORDER BY department_id ASC,  
        last_name DESC;
```



Pratique WHERE, operadores lógicos e ORDER BY combinando consultas reais sobre a tabela EMPLOYEES do schema HR.

- 1 Liste sobrenome e salário dos funcionários com salário entre 5000 e 12000 (use BETWEEN).
- 2 Mostre os funcionários cujos cargos sejam IT_PROG, SA_REP ou ST_MAN (use IN).
- 3 Liste todos os funcionários cujo sobrenome comece com a letra "K" (use LIKE).
- 4 Mostre os funcionários que NÃO recebem comissão (use IS NULL).
- 5 Liste todos os funcionários ordenados por department_id ASC e, dentro de cada departamento, por salário DESC.

Recapitulando

Nesta aula aprendemos a filtrar e ordenar dados em SQL. Pontos essenciais:

- WHERE filtra linhas conforme uma condição lógica
- Comparações simples: =, <, >, <=, >=, <>
- BETWEEN, IN e LIKE simplificam filtros complexos
- NULL exige IS NULL ou IS NOT NULL — nunca = NULL
- AND, OR e NOT combinam condições; lembre-se da precedência
- ORDER BY classifica o resultado: ASC ou DESC, por coluna, alias ou várias colunas